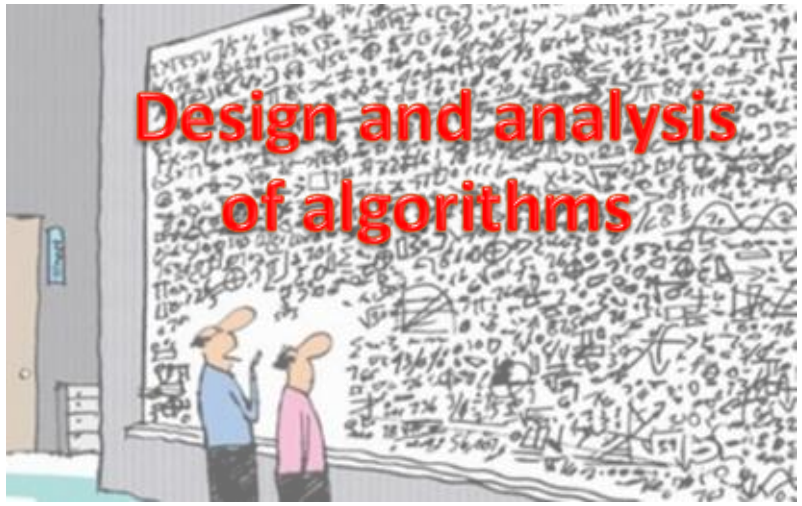


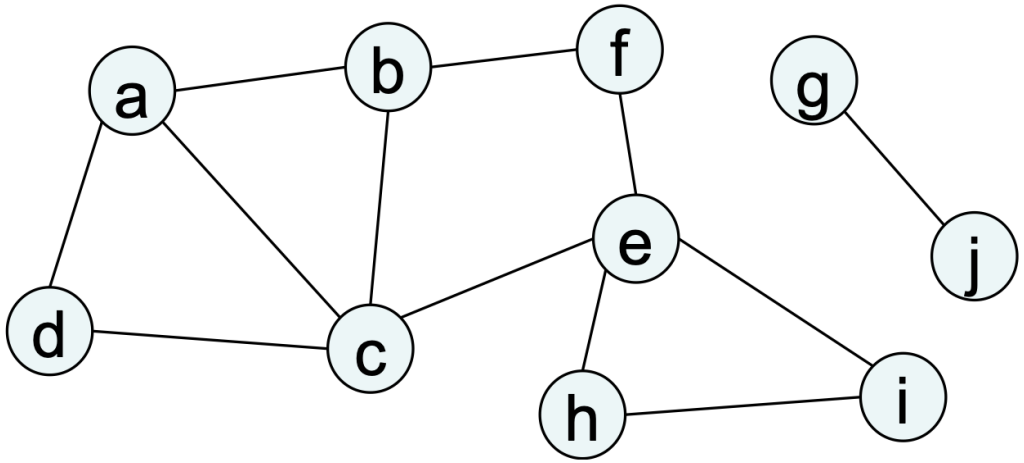
- **Breath-first search**

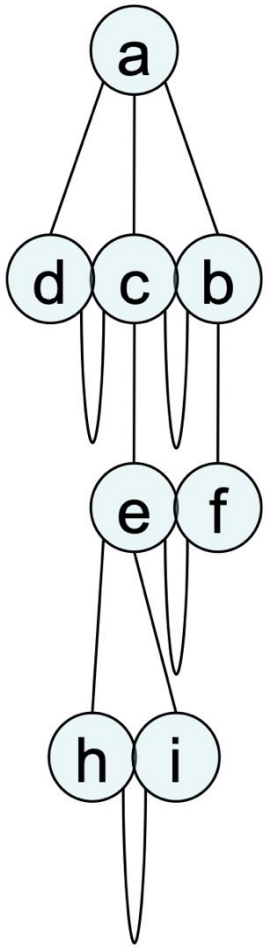
Objective

- Two BFS algorithms: queue, frontier implementation. implementation.
- The shortest path tree.
- Read DPV Chapter 4.1-4.2.
- More on trees.

Connecting nodes with strings of length 1

- "Lift" the graph by holding starting state a





distance 0

distance 1

distance 2

distance 3

can we have back edges?

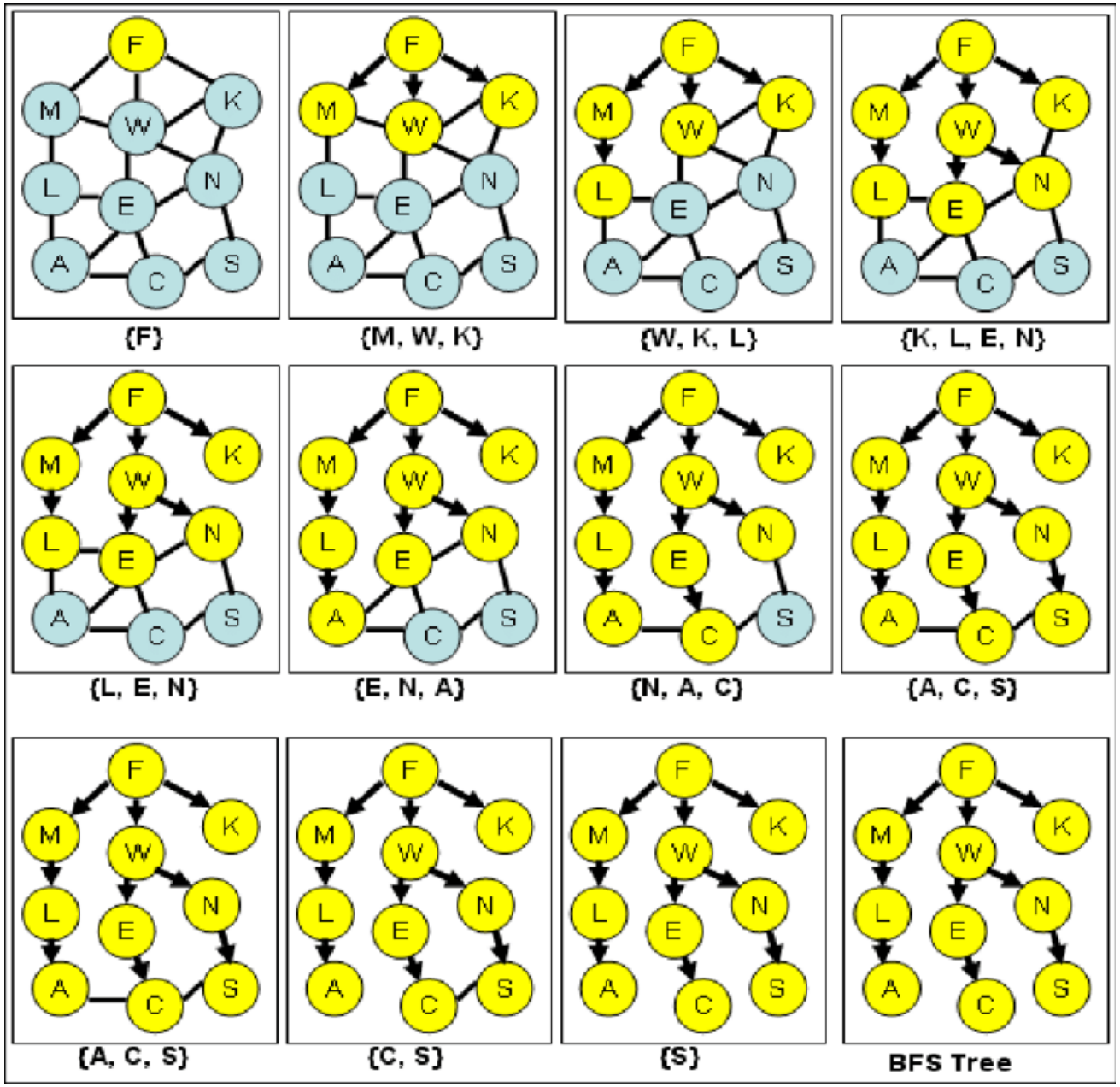
```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for each v ∈ G.Adj[u]
13            if v.color == WHITE
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)
18       u.color = BLACK
```

unprocessed

for processing

for processing

done, part of the tree



BFS (Frontier implementation)

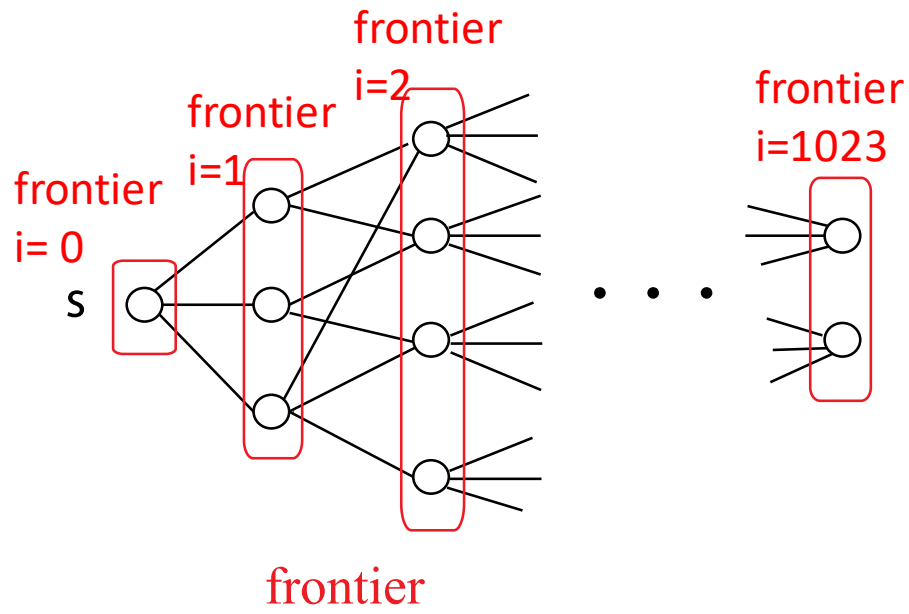
```
BFS(s,Adj):
    level={s:0}
    parent={s:None}
    i=1
    frontier=[s]      # previous level i-1
    while frontier:
        next=[]       # next level i
        for u in frontier
            for v in Adj[u]
                if v not in level
                    level[v]=i
                    parent[v]=u
                    next.append(v)
        frontier=next
        i+=1
```

BFS for some initial state s

level: stores the nodes seen so far and their minimum distance (i.e., level) from the root

parent: stores the nodes seen so far and their parent node

frontier: set of nodes with same level value i from which we explore the new nodes with level val. $= i + 1$



Conclusions

- Graphs: fundamental data structure
  - Directed and undirected
  - 4 possible representations
- Basic methods of graph search: BFS and DFS
- BFS: useful to find reachable nodes
  - Constructs the “reachability” or “shortest path” tree: each node knows who is the next node on his shortest path (minimum hop count) towards the root node